Grok filter plugin

Stack

- Plugin version: v4.4.4 (Other versions)
- Released on: 2026-02-11
- Changelog

Getting help

For questions about the plugin, open a topic in the Discuss forums. For bugs or feature requests, open an issue in Github. For the list of Elastic supported plugins, please consult the Elastic Support Matrix.

Description

Parse arbitrary text and structure it.

Grok is a great way to parse unstructured log data into something structured and queryable.

This tool is perfect for syslog logs, apache and other webserver logs, mysql logs, and in general, any log format that is generally written for humans and not computer consumption.

Logstash ships with about 120 patterns by default. You can find them here: https://github.com/logstash-plugins/logstash-patterns-core/tree/master/patterns. You can add your own trivially. (See the `patterns_dir` setting)

If you need help building patterns to match your logs, try the Grok debugger in Kibana.

Grok or Dissect? Or both?

The `dissect` filter plugin is another way to extract unstructured event data into fields using delimiters.

Dissect differs from Grok in that it does not use regular expressions and is faster. Dissect works well when data is reliably repeated. Grok is a better choice when the structure of your text varies from line to line.

You can use both Dissect and Grok for a hybrid use case when a section of the line is reliably repeated, but the entire line is not. The Dissect filter can deconstruct the section of the line that is repeated. The Grok filter can process the remaining field values with more regex predictability.

Grok Basics

Grok works by combining text patterns into something that matches your logs.

The syntax for a grok pattern is `%{SYNTAX:SEMANTIC}`

The `SYNTAX` is the name of the pattern that will match your text. For example, `3.44` will be matched by the `NUMBER` pattern and `55.3.244.1` will be matched by the `IP` pattern. The syntax is how you match.

The `SEMANTIC` is the identifier you give to the piece of text being matched. For example, `3.44` could be the duration of an event, so you could call it simply `duration`. Further, a string `55.3.244.1` might identify the `client` making a request.

For the above example, your grok filter would look something like this:

```
%{NUMBER:duration} %{IP:client}
```

Optionally you can add a data type conversion to your grok pattern. By default all semantics are saved as strings. If you wish to convert a semantic's data type, for example change a string to an integer then suffix it with the target data type. For example `%{NUMBER:num:int}` which converts the `num` semantic from a string to an integer. Currently the only supported conversions are `int` and `float`.

Examples: With that idea of a syntax and semantic, we can pull out useful fields from a sample log like this fictional http request log:

```
55.3.244.1 GET /index.html 15824 0.043
```

The pattern for this could be:

```
%{IP:client} %{WORD:method} %{URIPATHPARAM:request} %{NUMBER:bytes} %{NUMBER:duration}
```

A more realistic example, let's read these logs from a file:

```
input {
  file {
    path => "/var/log/http.log"
  }
}
filter {
  grok {
    match => { "message" => "%{IP:client} %{WORD:method} %{URIPATHPARAM:request} %{NUMBER:bytes} %{NUMBER:duration}" }
  }
}
```

After the grok filter, the event will have a few extra fields in it:

- `client: 55.3.244.1`
- `method: GET`
- `request: /index.html`
- `bytes: 15824`
- `duration: 0.043`

Regular Expressions

Grok sits on top of regular expressions, so any regular expressions are valid in grok as well. The regular expression library is Oniguruma, and you can see the full supported regexp syntax on the Oniguruma site.

Custom Patterns

Sometimes logstash doesn't have a pattern you need. For this, you have a few options.

First, you can use the Oniguruma syntax for named capture which will let you match a piece of text and save it as a field:

```
(?<field_name>the pattern here)
```

For example, postfix logs have a `queue id` that is an 10 or 11-character hexadecimal value. I can capture that easily like this:

```
(?<queue_id>[0-9A-F]{10,11})
```

Alternately, you can create a custom patterns file.

- Create a directory called `patterns` with a file in it called `extra` (the file name doesn't matter, but name it meaningfully for yourself)
- In that file, write the pattern you need as the pattern name, a space, then the regexp for that pattern.

For example, doing the postfix queue id example as above:

```
# contents of ./patterns/postfix:
POSTFIX_QUEUEID [0-9A-F]{10,11}
```

Then use the `patterns_dir` setting in this plugin to tell logstash where your custom patterns directory is. Here's a full example with a sample log:

```
Jan  1 06:25:43 mailserver14 postfix/cleanup[21403]: BEF25A72965: message-id=<20130101142543.5828399CCAF@mailserver14.example.com>
```

```
filter {
  grok {
    patterns_dir => ["./patterns"]
    match => { "message" => "%{SYSLOGBASE} %{POSTFIX_QUEUEID:queue_id}: %{GREEDYDATA:syslog_message}" }
  }
}
```

The above will match and result in the following fields:

- `timestamp: Jan 1 06:25:43`
- `logsource: mailserver14`
- `program: postfix/cleanup`
- `pid: 21403`
- `queue_id: BEF25A72965`
- `syslog_message: message-id=<20130101142543.5828399CCAF@mailserver14.example.com>`

The `timestamp`, `logsource`, `program`, and `pid` fields come from the `SYSLOGBASE` pattern which itself is defined by other patterns.

Another option is to define patterns *inline* in the filter using `pattern_definitions`. This is mostly for convenience and allows user to define a pattern which can be used just in that filter. This newly defined patterns in `pattern_definitions` will not be available outside of that particular `grok` filter.

Migrating to Elastic Common Schema (ECS)

To ease migration to the Elastic Common Schema (ECS), the filter plugin offers a new set of ECS-compliant patterns in addition to the existing patterns. The new ECS pattern definitions capture event field names that are compliant with the schema.

The ECS pattern set has all of the pattern definitions from the legacy set, and is a drop-in replacement. Use the `ecs_compatibility` setting to switch modes.

New features and enhancements will be added to the ECS-compliant files. The legacy patterns may still receive bug fixes which are backwards compatible.

Grok Filter Configuration Options

This plugin supports the following configuration options plus the Common options described later.

Setting	Input type	Required
break_on_match	boolean	No
ecs_compatibility	string	No
keep_empty_captures	boolean	No
match	hash	No
named_captures_only	boolean	No
overwrite	array	No
pattern_definitions	hash	No
patterns_dir	array	No
patterns_files_glob	string	No
tag_on_failure	array	No
tag_on_timeout	string	No
timeout_millis	number	No
timeout_scope	string	No

Also see Common options for a list of options supported by all filter plugins.

break_on_match

- Value type is boolean
- Default value is `true`

Break on first match. The first successful match by grok will result in the filter being finished. If you want grok to try all patterns (maybe you are parsing different things), then set this to false.

ecs_compatibility

- Value type is string
- Supported values are:
  - `disabled`: the plugin will load legacy (built-in) pattern definitions
  - `v1`, `v8`: all patterns provided by the plugin will use ECS compliant captures
- Default value depends on which version of Logstash is running:
  - When Logstash provides a `pipeline.ecs_compatibility` setting, its value is used as the default
  - Otherwise, the default value is `disabled`.

Controls this plugin's compatibility with the Elastic Common Schema (ECS). The value of this setting affects extracted event field names when a composite pattern (such as `HTTPD_COMMONLOG`) is matched.

keep_empty_captures

- Value type is boolean
- Default value is `false`

If `true`, keep empty captures as event fields.

match

- Value type is hash
- Default value is `{}`

A hash that defines the mapping of *where to look*, and with which patterns.

For example, the following will match an existing value in the `message` field for the given pattern, and if a match is found will add the field `duration` to the event with the captured value:

```
filter {
grok {
    match => {
      "message" => "Duration: %{NUMBER:duration}"
    }
  }
}
```

If you need to match multiple patterns against a single field, the value can be an array of patterns:

```
filter {
  grok {
    match => {
      "message" => [
        "Duration: %{NUMBER:duration}",
        "Speed: %{NUMBER:speed}"
      ]
    }
  }
}
```

To perform matches on multiple fields just use multiple entries in the `match` hash:

```
filter {
  grok {
    match => {
      "speed" => "Speed: %{NUMBER:speed}"
      "duration" => "Duration: %{NUMBER:duration}"
    }
  }
}
```

However, if one pattern depends on a field created by a previous pattern, separate these into two separate grok filters:

```
filter {
  grok {
    match => {
      "message" => "Hi, the rest of the message is: %{GREEDYDATA:rest}"
    }
  }
  grok {
    match => {
      "rest" => "a number %{NUMBER:number}, and a word %{WORD:word}"
    }
  }
}
```

`named_captures_only`

- Value type is boolean
- Default value is `true`

If `true`, only store named captures from grok.

`overwrite`

- Value type is array
- Default value is `[]`

The fields to overwrite.

This allows you to overwrite a value in a field that already exists.

For example, if you have a syslog line in the `message` field, you can overwrite the `message` field with part of the match like so:

```
filter {
  grok {
    match => { "message" => "%{SYSLOGBASE} %{DATA:message}" }
    overwrite => [ "message" ]
  }
}
```

In this case, a line like `May 29 16:37:11 sadness logger: hello world` will be parsed and `hello world` will overwrite the original message.

If you are using a field reference in `overwrite`, you must use the field reference in the pattern. Example:

```
filter {
  grok {
    match => { "somefield" => "%{NUMBER} %{GREEDYDATA:[nested][field][test]}" }
    overwrite => [ "[nested][field][test]" ]
  }
}
```

`pattern_definitions`

- Value type is hash
- Default value is `{}`

A hash of pattern-name and pattern tuples defining custom patterns to be used by the current filter. Patterns matching existing names will override the pre-existing definition. Think of this as inline patterns available just for this definition of grok

`patterns_dir`

- Value type is array
- Default value is `[]`

Logstash ships by default with a bunch of patterns, so you don't necessarily need to define this yourself unless you are adding additional patterns. You can point to multiple pattern directories using this setting. Note that Grok will read all files in the directory matching the patterns_files_glob and assume it's a pattern file (including any tilde backup files).

```
patterns_dir => ["/opt/logstash/patterns", "/opt/logstash/extra_patterns"]
```

Pattern files are plain text with format:

```
NAME PATTERN
```

For example:

```
NUMBER \d+
```

The patterns are loaded when the pipeline is created.

`patterns_files_glob`

- Value type is string
- Default value is `"*"`

Glob pattern, used to select the pattern files in the directories specified by patterns_dir

`tag_on_failure`

- Value type is array
- Default value is `["_grokparsefailure"]`

Append values to the `tags` field when there has been no successful match

`tag_on_timeout`

- Value type is string
- Default value is `"_groktimeout"`

Tag to apply if a grok regexp times out.

`target`

- Value type is string
- There is no default value for this setting

Define target namespace for placing matches.

`timeout_millis`

- Value type is number
- Default value is `30000`

Attempt to terminate regexps after this amount of time. This applies per pattern if multiple patterns are applied This will never timeout early, but may take a little longer to timeout. Actual timeout is approximate based on a 250ms quantization. Set to 0 to disable timeouts

`timeout_scope`

- Value type is string
- Default value is `"pattern"`
- Supported values are `"pattern"` and `"event"`

When multiple patterns are provided to `match`, the timeout has historically applied to *each* pattern, incurring overhead for each and every pattern that is attempted; when the grok filter is configured with `timeout_scope => event`, the plugin instead enforces a single timeout across all attempted matches on the event, so it can achieve similar safeguard against runaway matchers with significantly less overhead.

It's usually better to scope the timeout for the whole event.

Common options

These configuration options are supported by all filter plugins:

Setting	Input type	Required
`add_field`	hash	No
`add_tag`	array	No
`enable_metric`	boolean	No
`id`	string	No
`periodic_flush`	boolean	No
`remove_field`	array	No
`remove_tag`	array	No

`add_field`

- Value type is hash
- Default value is `{}`

If this filter is successful, add any arbitrary fields to this event. Field names can be dynamic and include parts of the event using the `%{field}`.

Example:

```
filter {
  grok {
    add_field => { "foo_%{somefield}" => "Hello world, from %{host}" }
  }
}
```

```
# You can also add multiple fields at once:
filter {
  grok {
    add_field => {
      "foo_%{somefield}" => "Hello world, from %{host}"
      "new_field" => "new_static_value"
    }
  }
}
```

If the event has field `"somefield" == "hello"` this filter, on success, would add field `foo_hello` if it is present, with the value above and the `%{host}` piece replaced with that value from the event. The second example would also add a hardcoded field.

`add_tag`

- Value type is array
- Default value is `[]`

If this filter is successful, add arbitrary tags to the event. Tags can be dynamic and include parts of the event using the `%{field}` syntax.

Example:

```
filter {
  grok {
    add_tag => [ "foo_%{somefield}" ]
  }
}
```

```
# You can also add multiple tags at once:
filter {
  grok {
    add_tag => [ "foo_%{somefield}", "taggedy_tag"]
  }
}
```

If the event has field `"somefield" == "hello"` this filter, on success, would add a tag `foo_hello` (and the second example would of course add a `taggedy_tag` tag).

`enable_metric`

- Value type is boolean
- Default value is `true`

Disable or enable metric logging for this specific plugin instance by default we record all the metrics we can, but you can disable metrics collection for a specific plugin.

`id`

- Value type is string
- There is no default value for this setting.

Add a unique `ID` to the plugin configuration. If no ID is specified, Logstash will generate one. It is strongly recommended to set this ID in your configuration. This is particularly useful when you have two or more plugins of the same type, for example, if you have 2 grok filters. Adding a named ID in this case will help in monitoring Logstash when using the monitoring APIs.

```
filter {
  grok {
    id => "ABC"
  }
}
```

`periodic_flush`

- Value type is boolean
- Default value is `false`

Call the filter flush method at regular interval. Optional.

`remove_field`

- Value type is array
- Default value is `[]`

If this filter is successful, remove arbitrary fields from this event. Fields names can be dynamic and include parts of the event using the %{field} Example:

```
filter {
  grok {
    remove_field => [ "foo_%{somefield}" ]
  }
}
```

```
# You can also remove multiple fields at once:
filter {
  grok {
    remove_field => [ "foo_%{somefield}", "my_extraneous_field" ]
  }
}
```

If the event has field `"somefield" == "hello"` this filter, on success, would remove the field with name `foo_hello` if it is present. The second example would remove an additional, non-dynamic field.

`remove_tag`

- Value type is array
- Default value is `[]`

If this filter is successful, remove arbitrary tags from the event. Tags can be dynamic and include parts of the event using the `%{field}` syntax.

Example:

```
filter {
  grok {
    remove_tag => [ "foo_%{somefield}" ]
```

```
  }
}
```

```
# You can also remove multiple tags at once:
filter {
  grok {
    remove_tag => [ "foo_%{somefield}", "sad_unwanted_tag"]
  }
}
```

If the event has field `"somefield" == "hello"` this filter, on success, would remove the tag `foo_hello` if it is present. The second example would remove a sad, unwanted tag as well.

Previous
geoip

Next
http